8

Das DOM-Konzept

8.1 Objekte und Hierarchie des DOM

Was ist DOM?

In JavaScript werden Ihnen neben den vordefinierten Objekten wie `String` und `Date` weitere Objekte zur Verfügung gestellt, über die Sie beispielsweise auf ein Element eines Formulars zugreifen und dieses ändern können. Diese Objekte gehören explizit nicht zu JavaScript, sondern zu der Objektschnittstelle **DOM** (**D**ocument **O**bject **M**odel). Diese Objektschnittstelle wird vom W3C (World Wide Web Consortium) standardisiert (*http://www.w3c.org*) und ist aus beliebigen Sprachen zugänglich, wenn diese im Rahmen eines Browsers verfügbar sind (beispielsweise VBScript, Java, JScript, ECMAScript, JavaScript).

Diese Objekte des DOM sind in mehreren Hierarchiestrukturen geordnet. Die hierarchiehöchsten Objekte sind …

- das Fensterobjekt `window`,
- das Navigatorobjekt `navigator` und
- das Bildschirmobjekt `screen`.

Entwicklung von DOM

Das DOM-Konzept wurde ursprünglich von Netscape entwickelt, aber bereits 1997 vom W3C übernommen und standardisiert. Es gibt verschiedene Level (Versionen), die jedoch unterschiedlich weit in Browsern implementiert sind. Derzeit kann man im Web Level 2 voraussetzen, wenn man nicht noch ganz alte Browser unterstützen muss. Techniken, die in neueren Versionen vom DOM (DOM 3 und das DOM-Konzept, das mit HTML5 verbunden ist) eingeführt wurden, sind derzeit in der Praxis immer noch mit Vorsicht und genauer Prüfung des Browsers eines Besuchers zu verwenden.

Mit der Entwicklung von XML (e**X**tensible **M**arkup **L**anguage) hat das W3-Konsortium das ursprüngliche DOM-Konzept modifiziert und universell auf baumartig strukturierte Dokumente anwendbar gemacht. Dieses Objektmodell bildet die Struktur eines baumartig strukturierten Dokuments im Arbeitsspeicher ab und erlaubt den Zugriff auf die Elementstruktur des Dokuments über eine API-Schnittstelle (API = **A**pplication **P**rogramming **I**nterface).

© HERDT-Verlag
Schulversion

Über bestimmte Methoden und Eigenschaften können somit die einzelnen Elemente, Attribute und Inhalte angesprochen, verarbeitet oder verändert werden. Sie können die Elemente, die Attribute und den Inhalt jeder HTML-Datei auslesen und löschen, aber auch neue Elemente erstellen und in das Dokument einfügen. Dies ist möglich, da das Document Object Model den Zugriff auf jedes Element eines HTML-Dokuments erlaubt. Somit haben Sie die Möglichkeit, Dokumente dynamisch zu generieren. Dazu wird die geladene Webseite im DOM in einzeln identifizierbare Elemente zerlegt und nach HTML-Tags, Attributen, Typen und der Position innerhalb der Webseite indiziert. Über diese Baumstruktur können Sie mit JavaScript Elemente ansprechen und deren Werte ändern.

Die hierarchiehöchsten Objekte und ihre Unterobjekte

Der Inhalt eines Fensters und somit das HTML-Dokument wird durch das `document`-Objekt repräsentiert. Dieses Objekt ist dem `window`-Objekt untergeordnet. Das HTML-Dokument enthält Elemente wie z. B. Grafiken, Verweise und Formulare.



Für diese Elemente gibt es teils weitere Unterobjekte, beispielsweise das Objekt `forms` für Formulare. Um auf die Elemente des Formulars (Eingabefelder, Auswahllisten oder Schaltflächen) zuzugreifen, existiert das Unterobjekt `elements`, mit dem Sie einzelne Felder und andere Elemente innerhalb eines Formulars ansprechen können.

Für das `navigator`-Objekt und das `screen`-Objekt existieren Hierarchien, die unabhängig von der Hierarchie des `window`-Objekts sind. Diese Objekte geben Auskunft über den Browser, in dem das Skript läuft, bzw. über die Einstellungen des Computerbildschirms eines Besuchers.

Aufgrund der großen Menge an Objekten im DOM und deren zahlreicher Eigenschaften und Methoden werden Ihnen in diesem Kapitel wie auch in den folgenden Kapiteln anhand von ausgesuchten Beispielen die wichtigsten Objekte und deren Eigenschaften und Methoden erläutert.

Plus **Ergänzende Lerninhalte:** *Übersichten über die Objektmodelle.pdf*
JavaScript-Techniken für ältere Skripte.pdf

Hier finden Sie z. B. weitere Ausführungen zu DOM-Techniken, die man früher oft verwendet hat, die aber mittlerweile an Bedeutung verloren haben oder nicht mehr zu empfehlen sind. Wenn Sie allerdings alte JavaScripts sehen, kann es sein, dass ein Verständnis dieser alten Vorgehensweisen nützlich ist.

8.2 Das Objekt `window`

Das Objekt `window` (Fenster) steht in der Objekthierarchie des DOM an oberster Stelle.
Alle anderen Objekte, die das im Fenster geladene Dokument beschreiben, sind Eigenschaften des `window`-Objekts. Es erlaubt Ihnen somit Abfragen zu einzelnen Dokumentfenstern und ermöglicht Ihnen, neue Fenster zu öffnen sowie deren Eigenschaften (Name, Höhe, Breite usw.) festzulegen. Auch das Schließen von Fenstern ist hiermit möglich.

© HERDT-Verlag

- Ist das Browserfenster in sogenannte Frames unterteilt, befinden sich die Objekte der HTML-Dokumente in einer eigenen Hierarchie unterhalb eines `frames`-Objekts. Frames sind allerdings veraltet und werden – bis auf IFrames – so gut wie gar nicht mehr eingesetzt.
- Das `document`-Objekt enthält alle wesentlichen HTML-Elemente wie Links, Anker, Bilder und Formulare des betreffenden HTML-Dokuments.
- Über das `event`-Objekt können Sie Ereignisse im Browser abfragen (Klicken, Auswählen usw.) und darauf reagieren.
- Das `history`-Objekt enthält die Liste der zuletzt geladenen URLs.
- Das `location`-Objekt enthält Informationen über die URL des aktuellen Dokuments.

Zusätzlich existieren weitere untergeordnete Objekte mit Eigenschaften, die in den folgenden Abschnitten im Einzelnen besprochen werden.

Methoden des `window`-Objekts

Das `window`-Objekt besitzt eine ganze Reihe an Methoden, von denen in der folgenden Tabelle die wichtigsten angegeben werden sollen.

Methode	Bedeutung
`alert()`	Ein kleines Meldungsfenster. Es wurde früher oft verwendet, wird aber mittlerweile in Webseiten nicht mehr eingesetzt. Programmierer nutzen es jedoch immer noch gerne zur Fehlersuche ohne Debugger.
`blur()`	Diese Methode deaktiviert das betreffende Browserfenster. Es wird in den Hintergrund verschoben.
`clearTimeout(TimeoutID)`	Der Zeitgeber, der mit `TimeoutID = setTimeout()` gestartet wurde, wird angehalten.
`close()`	Damit schließen Sie ein Browserfenster.
`confirm()`	Es wird ein Bestätigungsfenster angezeigt.
`focus()`	Hiermit aktivieren Sie das Browserfenster. Es wird in den Vordergrund gebracht.
`open()`	Mit der Methode `open()` öffnen Sie ein neues Browserfenster.
`prompt()`	Zur Eingabe eines Textes wird ein Eingabefenster angezeigt. Es wurde früher oft verwendet, wird aber mittlerweile in der Praxis nicht mehr eingesetzt. Eingaben werden nur noch über Formulare vorgenommen.
`setTimeout()`	Nach der in Millisekunden angegebenen Zeit wird ein festgelegter JavaScript-Ausdruck ausgeführt. Als Rückgabewert der Methode wird eine `TimeoutID` geliefert, mit der sich der laufende Zeitgeber referenzieren lässt.

© HERDT-Verlag
Schulversion

Eigenschaften des `window`-Objekts

Die Eigenschaften des `window`-Objekts besitzen in der Praxis so gut wie keine Bedeutung mehr.

Standardmäßig müssen Sie zum Aufruf einer Methode oder Eigenschaft eines Objekts den Objektnamen voranstellen, z. B. `window.alert()` oder `window.document`. Im Falle des `window`-Objekts können Sie den Namen `window` weglassen. Dieser wird automatisch von JavaScript vorangesetzt.

Fenster öffnen und schließen

Beim Browsen im Internet wurden in der Vergangenheit bei vielen Webseiten automatisch weitere Fenster, meist Werbeeinblendungen, geöffnet (sogenannte **Pop-ups**). Dazu wurde meist die Methode `open()` verwendet.

```
window.open("URL", "Fenstername", [Optionen])
```

- Die angegebene URL wird in einem Fenster mit den angegebenen Fensternamen geöffnet. Geben Sie eine leere Zeichenkette an, wird eine leere Seite angezeigt.
- Damit Sie die Inhalte von geöffneten Fenstern ansprechen können, müssen Sie dem Fenster einen eindeutigen Namen geben.
- Über weitere Optionen können Sie die Anzeige des Fensters konfigurieren.
- Als Rückgabewert wird eine Referenz auf das `window`-Objekt des erstellten Fensters geliefert.

Durch den starken Missbrauch dieser Techniken hat sich der Einsatz von Pop-up-Blockern durchgesetzt, und moderne Browser verhindern in der Grundeinstellung zudem das Öffnen und Manipulieren von Fenstern durch Skripte. Die Anwendung der Methode `open()` und weiterer Methoden zur Manipulation von Fenstern ist daher in der Praxis so gut wie gar nicht mehr zu finden und grundsätzlich nicht mehr zu empfehlen.

Den Zeitgeber (Timer) verwenden

Mit der Methode `setTimeout()` haben Sie die Möglichkeit, eine Anweisung oder Funktion nach einer bestimmten Verzögerungszeit auszuführen. Dazu geben Sie innerhalb der Klammern den entsprechenden JavaScript-Code an. Als zweite Angabe folgt die Verzögerungszeit in Millisekunden.

```
window.setTimeout("Anweisung/Funktion/Funktionsreferenz",
                  Millisekunden)
```


© HERDT-Verlag

Beispiel: *kap08/objekt_window_settimeout.html*

Durch die Anweisung `setTimeout("zeitVorbei()", 5000)` wird nach einer Zeit von 5 Sekunden (5.000 Millisekunden) die Funktion `zeitVorbei()` aufgerufen.

```
<body>
  <h3>Bitte warten, der Countdown läuft</h3>
  <script type="text/javascript">
    function zeitVorbei()
    {
      alert("Die Zeit ist abgelaufen...");
      return;
    }
    setTimeout(zeitVorbei, 5000);
  </script>
</body>
```

Beachten Sie, dass der erste Parameter eine Funktionsreferenz (also ohne runde Klammern) ist und kein Funktionsaufruf. Sie können hier auch einen Funktionsaufruf notieren (dann aber in Hochkommata eingeschlossen), aber die Funktionsreferenz ist besser für die Ressourcen und zudem sauberer in das Konzept der allgemeinen Ereignisbehandlung in JavaScript integriert.

Ein Beispiel für ein Meldungsfenster

Obwohl diese Art der Meldungsfenster veraltet ist, soll ein kurzes Beispiel die grundsätzliche Verwendung von `alert()` (Alarm) zeigen, da man dies zur Fehlersuche verwenden kann. Es zeigt eine Information an, die der Benutzer mit einem Klick auf die Schaltfläche *Ok* bestätigen muss. Als Parameter können Sie eine Zeichenkette oder Variable angeben, die als Meldung angezeigt werden soll.

```
alert("Anzeige");
```

Ein Beispiel für ein Bestätigungsfenster

Über das `confirm()`–Dialogfenster (Bestätigung) können Sie eine Frage anzeigen, die der Benutzer über die Schaltflächen *Ok* und *Abbrechen* beantworten kann. Als Parameter geben Sie die anzuzeigende Frage an.

```
confirm("Frage");
```

Über den Rückgabewert können Sie die betätigte Schaltfläche auswerten. Es werden die folgenden Rückgabewerte geliefert:

- `true`, wenn der Benutzer die Frage mit *Ok* bestätigt hat,
- `false`, wenn er die Frage verneint, d. h., die Schaltfläche *Abbrechen* betätigt.

Wichtig ist, dass der Anwender das Dialogfenster erst bestätigen muss, bevor das Skript fortgesetzt wird.

Beispiel: *kap08/interaktion_confirm.html*

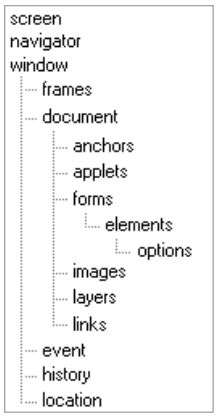
In Abhängigkeit von der Beantwortung der Frage soll eine entsprechende Meldung angezeigt werden.

```
<body>
  <script type="text/javascript">
    wahl = confirm("Sind Sie das erste Mal hier?");
    if(wahl == true)
      alert("Sie haben die Frage mit OK bestätigt!")
    else
      alert("Sie haben die Frage mit ABBRECHEN beantwortet!");
  </script>
</body>
```



Ein Bestätigungsdialog

8.3 Grundsätzliches zur Struktur des DOM einer Webseite



Die Struktur eines **Dokuments** im Browser ist aus Sicht des DOM-Konzepts eine Art **Baum**, der beim Laden eines Dokuments durch den Browser im Speicher aufgebaut wird. Beim Verlassen der Seite wird der Baum wieder gelöscht.

Ähnliche Elemente werden beim Aufbau des Baums bei der Indizierung auf gemeinsame dynamische Datenstapel (so genannte Stacks) für das Dokument abgelegt, die aus JavaScript über (besondere) Arrays bereitstehen. Auf diese Weise hat ein Browser nach dem Laden der Webseite genaue Kenntnis über alle relevanten Daten sämtlicher eigenständig für ihn ansprechbarer Elemente (Objekte) in der Webseite. Welche das sind und was er damit anstellen kann, das hat sich in der Vergangenheit je nach Browser erheblich unterschieden. Diese Unterschiede nehmen zwar ab, aber so ganz sind sie immer noch nicht verschwunden.

© HERDT-Verlag

Knoten, Verwandtschaftsbeziehungen und das `node`-Objekt

Die im Baum einer Webseite enthaltenen Elemente, aber auch andere Bestandteile eines Dokuments wie Attribute, Kommentare oder Textinhalte werden **Knoten (nodes)** genannt. Knotenstrukturen in einem Baum erlauben auch Arten von Beziehungen zu anderen Knoten zu definieren.

- Jeder Baum entspringt einem **Wurzelknoten** (**root**). Bei einer Webseite ist das das Element `html`.
- Jeder Teilbaum hat für seine spezielle Struktur einen entsprechenden Wurzelknoten, etwa bei einer Tabelle das `table`-Element oder bei einer Aufzählungsliste das Element `ul`.
- Untergeordnete Knoten werden in einem Baum als **Kinder** (children) bzw. **Kindelemente** oder – in tieferen Ebenen – als Nachfolger bezeichnet. Knoten auf derselben Ebene sind Geschwister (siblings) und übergeordnete Knoten Eltern (parent) bzw. Vorfahren, wenn es um mehrere Ebenen geht.
- Ausgehend vom Wurzelknoten, ist der Weg zu jedem anderen Knoten im Baum über diese Verwandtschaftsbeziehungen beschreibbar.

Wenn ein Browser die Webseite als ein strukturiertes Dokument aufgebaut hat, steht Ihnen damit ein sogenanntes `node`-Objekt zur Verfügung. Über dessen Eigenschaften und Methoden können Sie auf den Elementen im Baum navigieren und Teile einer Webseite dynamisch auswerten und ergänzen.

Dazu benötigen Sie Zugriff auf den jeweiligen Knoten, der Ihnen im DOM-Konzept **indirekt** bereitgestellt wird. Sie können **nicht** `node` als Objektnamen nehmen, sondern verwenden den Knoten, den die nachfolgend genauer vorgestellten Methoden `document.getElementById()` und `document.getElementsByName()` sowie `document.getElementsByTagName()` liefern. Zusätzlich werden die Knoten von Elementen in **Objektfeldern** bereitgestellt. Sie erhalten auf jedem Weg des Zugriffs das gleiche Objekt. Damit können Sie in jeder Webseite die Strukturen abfragen und gezielt verwenden.

Wichtige Knotentypen bei Webseiten

Das DOM-Konzept stellt eine ganze Reihe an Knotentypen zur Verfügung, die nur teilweise in Webseiten eine Rolle spielen (dafür aber etwa unter XML). Die wichtigsten Knotentypen für Webseiten im DOM-Konzept sind folgende:

- Ein **Dokumentknoten** repräsentiert die gesamte Baumstruktur einer Webseite.
- Ein **Elementknoten** entspricht exakt einem Element in HTML.
- Ein **Attributknoten** entspricht einem Attribut in HTML.
- Ein **Textknoten** repräsentiert den textuellen Inhalt eines Elements, wenn dieses einen solchen haben kann (etwa eine Überschrift oder ein `div`-Bereich).
- Ein **Kommentarknoten** repräsentiert jeden Bereich mit einem HTML-Kommentar.

Die meisten Elemente einer Webseite können bei Bedarf während der „Lebenszeit“ der Webseite aktualisiert werden (ein `<br />`-Element z. B. wäre nicht aktualisierbar). Etwa wenn mittels eines Skripts die Position eines Elementes in der Webseite oder dessen Inhalt verändert oder über Style Sheets nach dem vollständigen Laden der Webseite das Layout eines Elements dynamisch umgewandelt wird.

© HERDT-Verlag Schulversion

Das DOM besteht also aus Knoten, die Referenzen auf ihre über- und untergeordneten Knoten haben. Jeder Knoten repräsentiert genau ein Element oder Attribut in der Baumstruktur des Dokuments.

- Das aktuelle HTML-Dokument ist ein Dokumentknoten.
- Jedes HTML-Element ist ein Elementknoten.
- Texte in HTML-Elementen sind Textknoten.
- Jedes HTML-Attribut ist ein Attributknoten.
- Kommentare sind Kommentarknoten.

So besteht beispielsweise die HTML-Angabe

```
<div>Inhalt des Blocks</div>
```

aus zwei Knoten, dem umschließenden Elementknoten `div` und dem Textknoten "`Inhalt des Blocks`". Da sich der Textknoten innerhalb des Elements `div` befindet, wird er auch als Kindknoten (child node) des Elements `div` bezeichnet. Das umschließende Element wird Elternknoten (parent node) genannt. Befindet sich innerhalb des Elements `div` ein weiteres Element, wie beispielsweise

```
<div>Inhalt <i>des Blocks</i></div>
```

enthält das Element `div` zwei Kindknoten, das Wort `Inhalt` sowie das Element `i`. Der Text "`des Blocks`" ist nun nicht mehr das Kind des Elements `div`, sondern das Kind des Elements `i`.

Attributknoten (attribute nodes) entstehen, wenn bei einem Element ein Attribut angegeben wird.

```
<div style="text-align:center;">Inhalt <i>des Blocks</i></div>
```

Das Element `div` enthält nun drei Knoten, denn das Attribut `style` ist ein Attributknoten.

Eine Webseite dynamisch aus Knoten aufbauen

Eine Webseite kann per JavaScript dynamisch aus Knoten aufgebaut werden. Alle `node`-Objekte besitzen dazu geeignete Methoden und Eigenschaften, von denen nur die wichtigsten hier vorgestellt werden:

- Um Knoten an bestehende Knoten dynamisch anzufügen, besitzen alle Dokument- und Elementknoten eine Methode `appendChild()`. Der Name ergibt sich aus oben beschriebener Verwandtschaftsbeziehung, die man in einem Knotenbaum verwendet.
- Attributknoten werden mit `setAttributeNode()` einem Elementknoten angefügt.
- Den Zugriff auf den Wert eines Attributknotens können Sie mit der Eigenschaft `nodeValue` durchführen.

8.4 Das Objekt `document`

Nachfolgend finden Sie die Eigenschaften und Methoden des `document`-Objekts, das die Webseite selbst repräsentiert und in der Praxis eines der wichtigsten Objekte im DOM-Konzept ist.

© HERDT-Verlag

Die Eigenschaften

Das `document`-Objekt enthält eine Vielzahl Eigenschaften, über die Sie auf die Elemente eines HTML-Dokuments zugreifen. Viele Eigenschaften des Objekts sind veraltet, weil Sie HTML-Attribute repräsentieren, die heutzutage nicht mehr verwendet werden, etwa Eigenschaften für das Setzen von Farben über generierte HTML-Attribute – stattdessen verwendet man CSS. Diese Eigenschaften werden hier nicht mehr betrachtet. Die Tabelle enthält nur Eigenschaften, die in der Praxis noch eingesetzt werden.

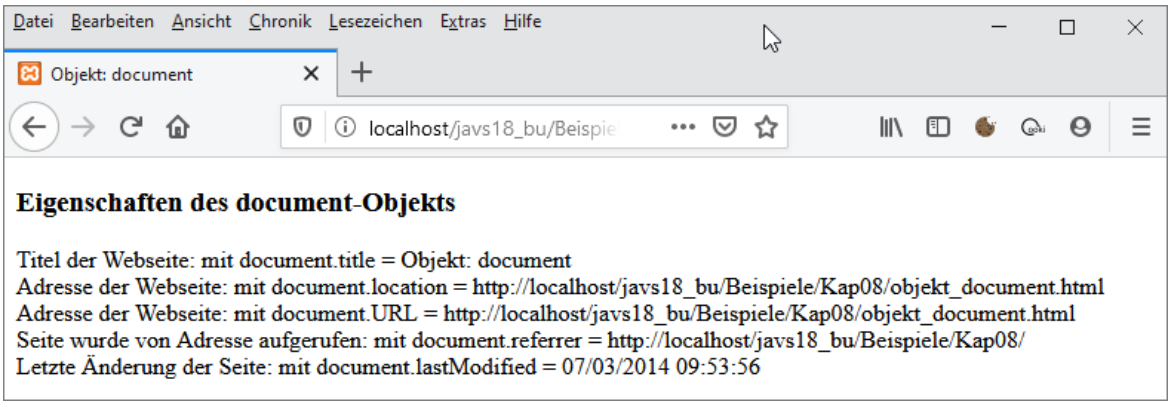
Eigenschaften	Bedeutung
`title`	Mit der Eigenschaft `title` lässt sich der Dokumenttitel auslesen bzw. setzen.
`referrer`	Die URL des Dokuments, das per Hyperlink auf das aktuelle Dokument verwiesen hat. Der Referrer wird nur übermittelt, wenn sich die Webseiten auf einem Webserver befinden und der Browser das nicht deaktiviert hat.
`lastModified`	Damit erhalten Sie das Datum, an dem das Dokument zuletzt verändert wurde.
`cookie`	Zugriff auf Cookies, die lokal abgespeichert werden und Parameter-Paare in Form von Zeichenketten enthalten können. Cookies können gelesen und verändert werden.
`URL`	Hiermit erhalten Sie die URL des geladenen Dokuments.

Die Eigenschaft `URL` ist ein Synonym für die Eigenschaft `document.location`, um bei der Anwendung eine Verwechslung mit `window.location` zu vermeiden. Denn `window.location` beinhaltet die vom Nutzer angeforderte URL, und `document.URL` beinhaltet die an ihn zurückgelieferte URL. Diese können sich durch Weiterleitungen (Redirects) voneinander unterscheiden.

Beispiel: *kap08/objekt_document.html*

In dem folgenden Beispiel werden die Werte von Eigenschaften von `document` ausgegeben. Dazu kommt die Methode `document.write()` zum Einsatz.

```
…
<body>
  <h3>Eigenschaften des document-Objekts</h3>
  <script type="text/javascript">
    document.write("Titel der Webseite: mit document.title = " +
                   document.title + "<br />");
    document.write("Adresse der Webseite: mit document.location = " +
                   document.location + "<br />");
    document.write("Adresse der Webseite: mit document.URL = " +
                   document.URL + "<br />");
    document.write("Seite wurde von Adresse aufgerufen:
                   mit document.referrer = " +
                   document.referrer + "<br />");
    document.write("Letzte Änderung der Seite:
                   mit document.lastModified = " +
                   document.lastModified + "<br />");
  </script>
</body>
</html>
```



Ausgabe von verschiedenen `document`-Eigenschaften

Die Methoden

Das `document`-Objekt enthält verschiedene Methoden, die in der Praxis teilweise nur selten eingesetzt werden. Andere Methoden sind dagegen unabdingbar für jede moderne Webseite. Hier folgt eine Auswahl wichtiger Methoden – ggf. mit einem Hinweis, ob diese wichtig sind oder nicht.

Methode	Bedeutung
`clear()`	Hiermit löschen Sie den Inhalt der aktuellen Webseite. Die Methode wird sehr selten eingesetzt.
`close()`	Die Methode `close()` schließt das Dokument, sodass keine weiteren Inhalte eingefügt werden können. Die Methode wird sehr selten eingesetzt.
`createAttribute()`	Erzeugen eines neuen **Attributknotens** für ein HTML-Element
`createElement()`	Erzeugen eines neuen **Elementknotens** im DOM-Baum eines Dokuments
`createTextNode()`	Mit der Methode können Sie im DOM-Baum eines Dokuments einen **Textknoten** erzeugen.
`getElementById()`	Damit können Sie auf ein HTML-Element mit einem im Attribut `id` festgelegten Namen zugreifen. Sie erhalten als Ergebnis den selektierten **Elementknoten**. Die Methode ist sehr wichtig und Bestandteil fast jeder modernen Webseite.
`getElementsByName()`	Die Methode funktioniert wie `getElementByTagName()`, nur dass hier das Attribut `name` eines HTML-Elements ausgewertet wird und alle Elemente **als Array** zurückgeliefert werden. Die Methode funktioniert nur bei einigen HTML-Elementen. Zuverlässig in allen Browsern ist nur der Zugriff auf die HTML-Elemente, die auch in den standardisierten Objektfeldern von `document` zur Verfügung stehen.

© HERDT-Verlag

Methode	Bedeutung
`getElementsByName()`	Alternativ kann man bei solchen Elementen auch direkt den Namen als Eigenschaft notieren, obwohl das nicht empfohlen wird und man in dem Fall die Methode `getElementByTagName()` verwenden sollte. Nachfolgend sehen Sie ein Beispiel mit gleichwertigen Zugriffen auf ein Formularfeld: `<form name="form1">` `…` `<input name="feld1" />` `…` `</form>` `<script type="text/javascript">` `document.write(document.form1.feld1.value + " ");` `document.write(document.getElememtsByName ("form1")[0].feld1.value + " ");` `document.write(document.getElememtsByName ("feld1")[0].value + " ");` `</script>`
`getElementsByTagName()`	Hiermit erhalten Sie ein Array der HTML-Elemente als Knoten, deren Namen Sie als Parameter angegeben haben, z. B. `h1`, `p`. Die Methode ist sehr wichtig und Bestandteil fast jeder modernen Webseite.
`open()`	Damit löschen Sie die aktuelle Webseite und öffnen das Dokument für neue Inhalte. Die Methode wird sehr selten eingesetzt.
`write(Text)` und `writeln(Text)`	Über diese Methoden schreiben Sie Text in das HTML-Dokument. Dieser wird an der Stelle eingefügt, an der die Methode aufgerufen wird. Bei `writeln(Text)` wird zusätzlich ein Zeilenumbruch eingefügt. Dieser Umbruch ist nur im Quelltext der Webseite sichtbar und entspricht nicht dem HTML-Tag ` `. Die Methoden werden heutzutage nur noch beim Laden einer Webseite eingesetzt. Die nachträgliche Veränderung einer Webseite, was bei dynamischen Webseiten ständig erfolgt, ist damit nicht adäquat möglich.

Beispiel: *kap08/elemente_erstellen.html*

In dem folgenden Beispiel werden Elemente der Webseite mit den Methoden von `document` und `node` dynamisch erstellt und einer Webseite hinzugefügt.

```
<body>
  <h1>Erstellen von DOM-Elementen</h1><hr />
  <script type="text/javascript">
    var bildquelle = document.createAttribute("src");
```

① ②

```
③     bildquelle.nodeValue = "images/b1.jpg";
④     var alternativtext = document.createAttribute("alt");
      alternativtext.nodeValue = "Am Fluss";
      var titel = document.createAttribute("title");
      titel.nodeValue = "An der Elbe";
⑤     var b1 = document.createElement("img");
⑥     b1.setAttributeNode(bildquelle);
      b1.setAttributeNode(alternativtext);
      b1.setAttributeNode(titel);
⑦     var u2 = document.createElement("h2");
      var text = document.createTextNode(
        "Erstellen von Knoten mit Methoden von document");
⑧     u2.appendChild(text);
⑨     document.getElementsByTagName("body")[0].appendChild(u2);
      document.getElementsByTagName("body")[0].appendChild(b1);
    </script>
</body>
```

① Zuerst sehen Sie in dem Quelltext der Webseite die HTML-Tags für eine Überschrift der Ordnung 1 und eine Trennlinie. Ansonsten enthält die Webseite keine weiteren Tags außer dem folgenden Skriptbereich.

② Mit der Methode `createAttribute()` wird ein Attributknoten erzeugt. Der Wert des Parameters ist `"src"`. Das ist der Name des Attributs für die Quelle eines Bildes bei einem `img`-Tag. Der erzeugte Attributknoten soll einem Bildobjekt hinzugefügt werden. Das Bildobjekt muss aber noch erzeugt werden. Der Attributknoten kann zu dem Zeitpunkt noch nicht dem Elementknoten für das noch zu erzeugende Bild angehängt oder sonst in die Webseite eingefügt worden sein. Die Referenz auf den Attributknoten wird in der Variablen `bildquelle` gespeichert und steht somit für die spätere Verwendung zur Verfügung.

③ Mit der Eigenschaft `nodeValue` wird der Wert des Attributknotens `bildquelle` gesetzt. Der Wert ist die relative Adresse einer Bilddatei, die in der Webseite angezeigt werden soll.

④ Mit der Methode `createAttribute()` werden zwei weitere Attributknoten erzeugt, denen dann jeweils mit `nodeValue` Werte zugewiesen werden. Die Attribute stehen für den Alternativtext und den Titel bei einem Bild.

⑤ Mit der Methode `createElement()` wird jetzt ein Elementknoten vom Typ `img` erstellt. Auch dieser Knoten ist danach noch nicht der Webseite hinzugefügt worden, die Referenz darauf steht in der Variablen `b1` zur Verfügung.

⑥ Bis zu dem Zeitpunkt gibt es noch keine Beziehung zwischen dem Elementknoten `b1` und den drei Attributknoten. Mit der Methode `setAttributeNode()` werden dem Elementknoten `b1` jetzt alle drei vorher erstellten Attributknoten zugewiesen. Die dynamisch generierte HTML-Struktur des Elementknotens `b1` sieht danach so aus:

```
<img src="images/b1.jpg" alt="Am Fluss" title="An der Elbe">
```

Der Knoten ist danach noch nicht der Webseite hinzugefügt worden und damit noch nicht sichtbar in der Webseite.

© HERDT-Verlag

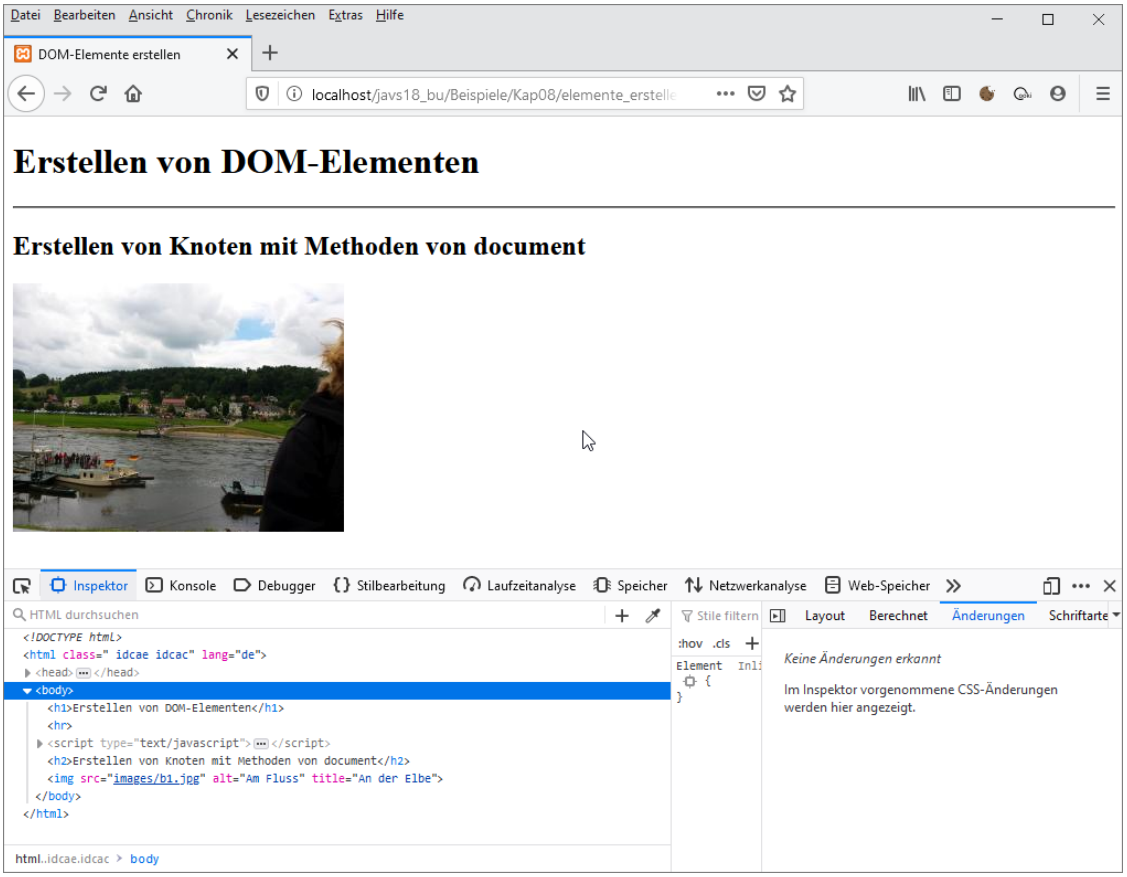
⑦ Mit der Methode `createElement()` wird ein Elementknoten vom Typ `h2` erstellt. Auch dieser Knoten ist nach dem Erstellen noch nicht der Webseite hinzugefügt worden, aber die Referenz darauf steht in der Variablen `u2` zur Verfügung. Dieses Element ist eine Überschrift der Ordnung 2, und die hat einen Textinhalt. Das bedeutet, dass das Kindelement dieses Elementknotens ein Textknoten ist, und so ein Textknoten wird im folgenden Schritt mit der Methode `createTextNode()` erstellt. Es gibt zu dem Zeitpunkt noch keine Verbindung zwischen dem Elementknoten `h2` und dem Textknoten `text`. In dem DOM der Webseite ist damit noch keine Überschrift der Ordnung 2 vorhanden.

⑧ In diesem Schritt wird der Textknoten `text` dem Elementknoten `u2` hinzugefügt. Dazu kommt die Methode `appendChild()` zu Einsatz. Die dynamisch generierte HTML-Struktur des Elementknotens sieht so aus:

```
<h2>Erstellen von Knoten mit Methoden von document</h2>
```

Der Knoten ist danach noch nicht der Webseite hinzugefügt worden und damit nicht sichtbar in der Webseite.

⑨ Mit der Methode `appendChild()` werden die Elementknoten `b1` und `u2` nacheinander der Webseite hinzugefügt. Die Webseite wird dazu mit `document.getElementsByTagName("body")[0]` selektiert. Beachten Sie, dass die Methode `document.getElementsByTagName()` ein Array zurückliefert und man mit dem Index unbedingt arbeiten muss. Beim `body`-Element, das in jeder Webseite nur einmal vorkommt, erscheint das überflüssig, aber da die Methode `document.getElementsByTagName()` **immer** ein Array liefert, muss mit dem Index gearbeitet werden. Die Knoten sind nun dynamisch der Webseite hinzugefügt worden und damit sichtbar.



Die dynamisch aufgebaute Webseite-Struktur, wie der Browser sie verwendet – Anzeige in der Web-Konsole / Inspektor

8.5 Zugriff auf Inhalte von Elementen in der Webseite

Der Zugriff auf den Inhalt von Elementen in einer Webseite per JavaScript lässt sich im Wesentlichen in drei Kategorien unterscheiden:

- Zugriff auf Textinhalte
- Zugriff auf Werte von Formularfeldern
- Zugriff auf klassische HTML-Attribute über deren DOM-Repräsentation

Zugriff auf Textinhalte mit `innerHTML`

Sie können bei jedem Element in der Webseite, das einen Text als Inhalt hat, diesen Textinhalt per JavaScript ansprechen und austauschen, sofern Sie den zugehörigen Knoten selektiert haben. Sie haben schon die Eigenschaft `nodeValue` gesehen, aber es gibt mit `innerHTML` eine einfachere Möglichkeit. Als Alternative steht `innerText` zur Verfügung.

Beide Eigenschaften gehörten schon lange zum Werkzeugkasten von JavaScript-Programmierern, obwohl sie nie zum offiziellen DOM-Standard gezählt haben. Dennoch funktioniert der Zugriff über `innerHTML` in allen neuen Browser-Generationen. In HTML5 wurde `innerHTML` dann auch offiziell eingeführt.

Beispiel: *kap08/elemente_innerhtml.html*

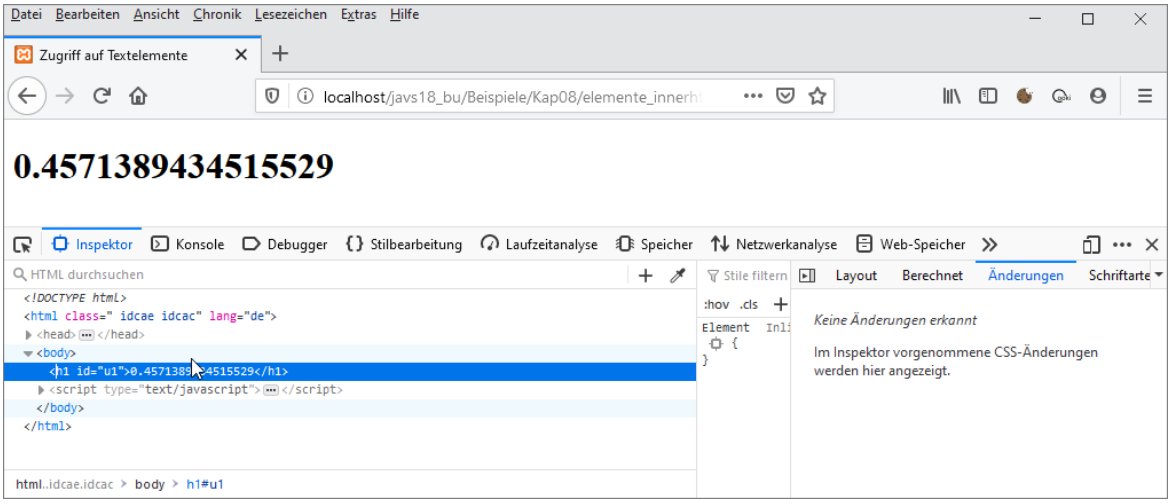
In dem folgenden Beispiel wird der Inhalt einer Überschrift über `innerHTML` dynamisch gefüllt.

```
<body>
①   <h1 id="u1"></h1>
    <script type="text/javascript">
②     document.getElementById("u1").innerHTML=Math.random();
    </script>
</body>
```

① Zuerst sehen Sie in der Webseite die HTML-Tags für eine leere Überschrift vom Typ `h1`. Diese Überschrift hat eine ID `u1`.

② Im Skript wird mit der Methode `document.getElementById()` die Überschrift selektiert und mit `innerHTML` der Inhalt für diesen Elementknoten gesetzt.
Mit `Math.random()` wird ein Zufallswert zwischen 0 und 1 in die Webseite geschrieben.

© HERDT-Verlag



Die dynamisch über `innerHTML` gefüllte Überschrift

Der Unterschied zwischen `innerHTML` und `innerText` ist gering. Über `innerHTML` wie auch `innerText` haben Sie Zugang zum Textinhalt eines HTML-Elements. Wenn Sie beim dynamischen Ändern des gespeicherten Inhalts bei `innerHTML` jedoch HTML-Tags notieren, werden diese bei der Aktualisierung des Elementinhalts interpretiert. Das ist bei `innerText` nicht der Fall. Allerdings wirkt sich das bei vielen Browsern nicht aus. Einige Browser sperren sogar die Verwendung von `innerText`. Auch sonst waren in der Vergangenheit diverse Probleme im Zusammenhang mit `innerText` bekannt, weshalb Sie in der Praxis `innerText` nicht verwenden sollten.

Der Zugriff auf `innerHTML` wird normalerweise nicht beim Laden einer Webseite erfolgen, sondern nach dem Laden der Webseite dynamisch aufgrund eines Ereignisses (vgl. Kapitel 9).

Zugriff auf klassische HTML-Attribute

Die per DOM verfügbaren Elemente einer Webseite verfügen in der Regel über die gleichen Eigenschaften, die als Attribute bei den zugrunde liegenden HTML-Tags vorhanden sind. Sie können also Kenntnisse aus HTML übernehmen und die Namen der bekannten Attribute von Elementen als Eigenschaften des zugehörigen Elementknotens verwenden. Indem Sie die Namen der Eigenschaften angeben, können Sie lesend und meist auch schreibend darauf zugreifen.

Beispiel: *kap08/elemente_klassisch.html*

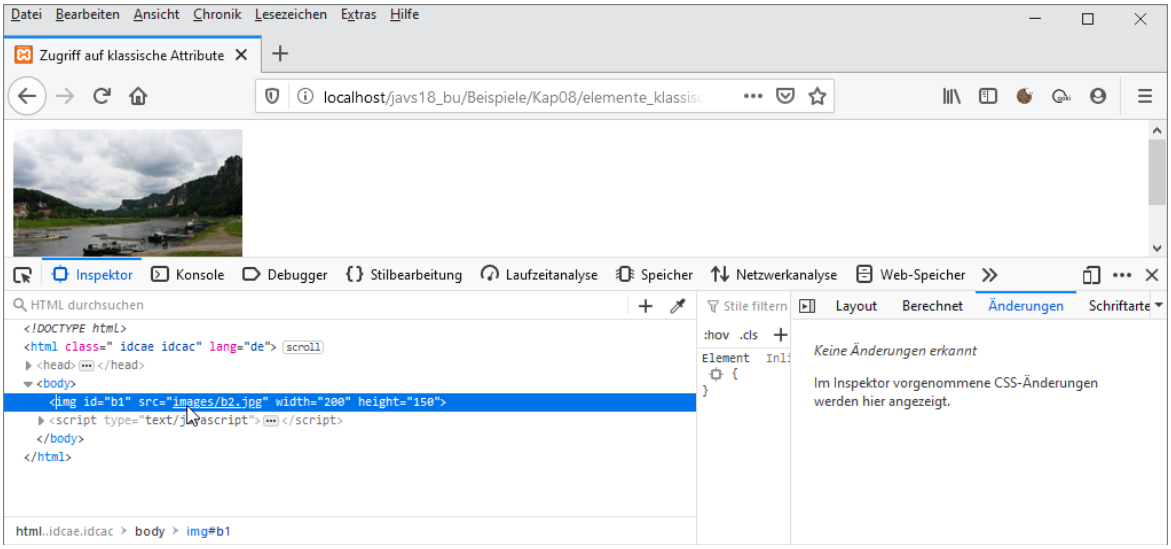
In dem folgenden Beispiel werden mehrere Attribute eines Bildes angesprochen.

```
<body>
  <img id="b1" />
  <script type="text/javascript">
    document.getElementById("b1").width = "200";
    document.getElementById("b1").height = "150";
    document.getElementById("b1").src = "images/b2.jpg";
  </script>
</body>
```

① ②

① Zuerst sehen Sie in der Webseite den HTML-Tag für eine leere Bildreferenz. Der Tag hat eine ID `b1`.

② Im Skript wird in den drei folgenden Anweisungen mit der Methode `document.getElementById()` das Bild selektiert. Die verwendeten Eigenschaften entsprechen den Namen klassischer HTML-Attribute bei diesem Tag.



Zugriff auf klassische HTML-Attribute

Zugriff auf Formularinhalte

Der Inhalt von Formulareingabefeldern steht über die Eigenschaft `value` zur Verfügung. Diese Eigenschaft ist in der Regel sowohl zu lesen als auch zu schreiben (vgl. Kapitel 10).

8.6 HTML-Elemente als Unterobjekte von `document`

Nun soll noch ein Blick auf die Unterobjekte von `document` aus Sicht der **Objektfelder** erfolgen. Ein `document`-Objekt besitzt Unterobjekte in Form standardisierter Arrays (Objektfelder), worüber auch ein Zugriff auf Elemente einer Webseite möglich ist. Wie bei sämtlichen generierten Arrays in JavaScript erhalten Sie über die Eigenschaft `length` die Anzahl der jeweils enthaltenen Objekte.

So können Sie über das Objektfeld `forms` auf Formulareigenschaften und -elemente zugreifen, sobald sich ein Formular (`<form>...</form>`) in einer Webseite befindet. Diese Art des Zugriffs war historisch die erste Möglichkeit, um Bestandteile einer Webseite aus JavaScript heraus anzusprechen. Heutzutage nutzt man sie nur noch sehr selten, aber grundsätzlich funktioniert diese Alttechnik immer noch in allen modernen Browsern, und es gibt einige Situationen, in denen diese historische Vorgehensweise sogar noch ihre Berechtigung hat. Ebenso ist wichtig, dass ein Zugriff über den Namen auf DOM-Elemente auf die Elemente Objektfelder in allen Browsern zuverlässig ist. Andere Elemente können nur in einigen Browsern so angesprochen werden.

© HERDT-Verlag